

Study of a Rotary Cylinder Controlled by a Proportional Valve with Control Implementation in an Electronic Simulation Environment

Autor: Simonca Raul

Cuprins

1 Introduction **3**
1.1 Objectives 3
1.2 Specifications 3

2 System Identification **4**
2.1 Data collection 4
2.2 Transfer function estimation 6

3 Electronic Simulation **7**
3.1 Design of the cylinder and proportional valve with the help operational amplifiers 7
3.2 Design of the position measurement 8
3.3 Control implementation of the electronic simulation 9

4 Test results **12**

1 Introduction

This project focuses on modeling and controlling a pneumatic cylinder actuated by a proportional valve. The goal is to identify a mathematical model (transfer function) that describes the dynamic behavior of the system and design a control strategy to achieve the desired performance. The control is implemented in an electronic simulated environment using analog components such as operational amplifiers and low-pass filters, simulating the rotary cylinder with proportional valve.

1.1 Objectives

The principal onjectives of the project are:

1. Identification of the rotary cylinder using the proportional valve.
2. Design of the electronic circuit for simulating the pneumatic setup.
3. Control implementation on the electronic circuit simulation.

1.2 Specifications

Using the *System Identification Toolbox* from the Matlab environment, the tranfer fucntion of the rotary cylinder can be identified.

In the simulation of the penumatic setup, there will be used multiple Opamp circuits.

For the control, a dev board made by me, using the *AVR32DA32* will be used.

2 System Identification

Every system can be described by a transfer function or by a state system matrix,by using parametric methods and applying linear regression on the output signal. Using the *System Identification Toolbox* we can hope to achieve that.

2.1 Data collection

To identify the dynamic model of the cylinder, experimental data was collected under controlled conditions. This involved varying the input voltage to the valve and measuring the corresponding movement of the rotary cylinder. The rotary cylinder was a **DSM/DSM-B** and the proportional valve was a **MPYE-5-1**. Using the in-house made evaluation board driving a **AVR32DA32** and running a self developed firmware, I was able to read the output of the semi-rotary cylinder by using a shunt resistor of 250 Ω. I choose this resistor based on Ohms's law that says $I = \frac{U}{R}$ and that my cylinder position sensor relays. The sensor value is between 4mA and 20 mA. From the Ohm's law, the voltage will be between 5v and 1v. In regards to safety a lower value of R was chosen for tests so the resulting max voltage would be lower than 5v.

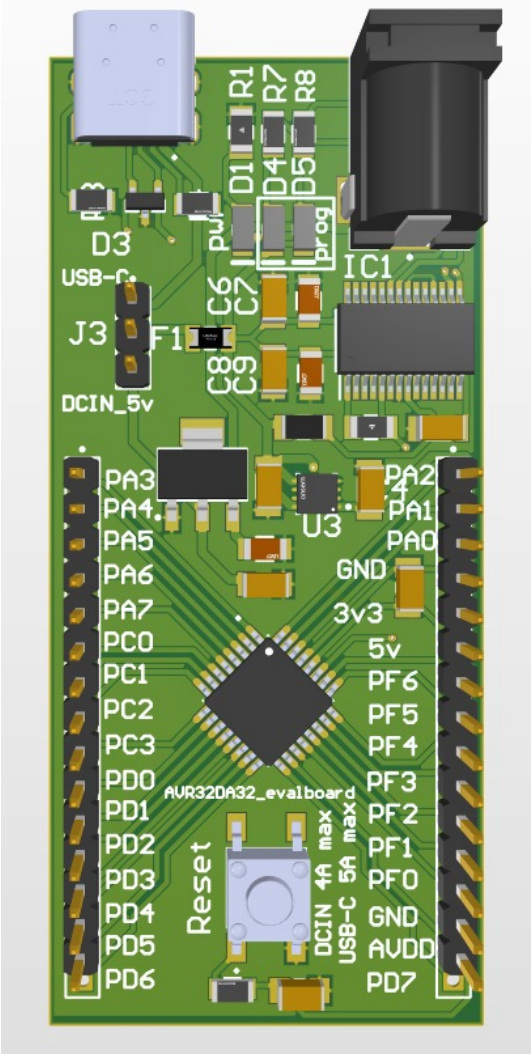


Figura 1: AvR32DA32 Evaluation Board

The board c an be programmed using the **pymcuprog** command,on the UPDI interface and it's recognized as a USB to UART device. The following graphs are the measured positions of the rotary cylinder where the command is between 0v and 10v. The datasheet of the valve specifies that the command has to be between 0v and 10v and that the direction clockwise of the semi-rotary cylinder is between 0v and 5v and the counter clock-wise is 5v and 10v. The lowest possible command of the valve, or the least amount of flow is around 5v, but because of some errors in wiring, for this tests, it's reversed.

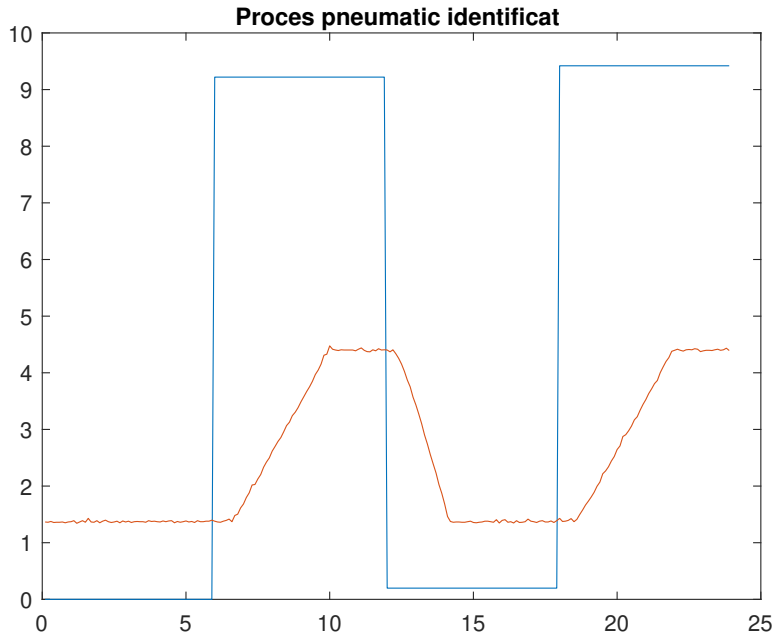


Figura 2: Identification data

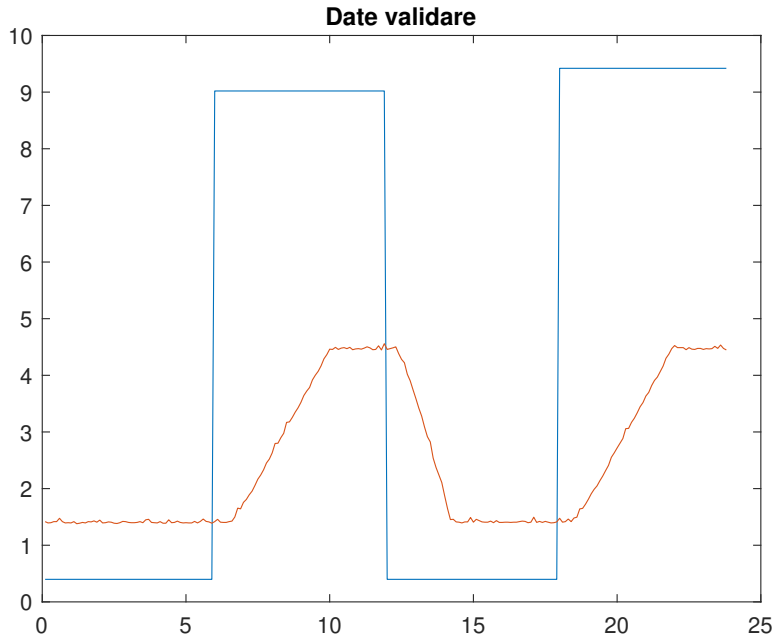


Figura 3: Validation data

2.2 Transfer function estimation

In matlab to get an object of type *iddata*, the output,input and sample time has to be passed to the constructor. After getting the iddata object, the **System Identification Toolbox** may be used. First data has to be imported, more specifically a *iddata* object has to be imported to be able to identify the parameters. After the object is imported the identification may begin and the resulting transfer function can be exported to the Matlab workspace.

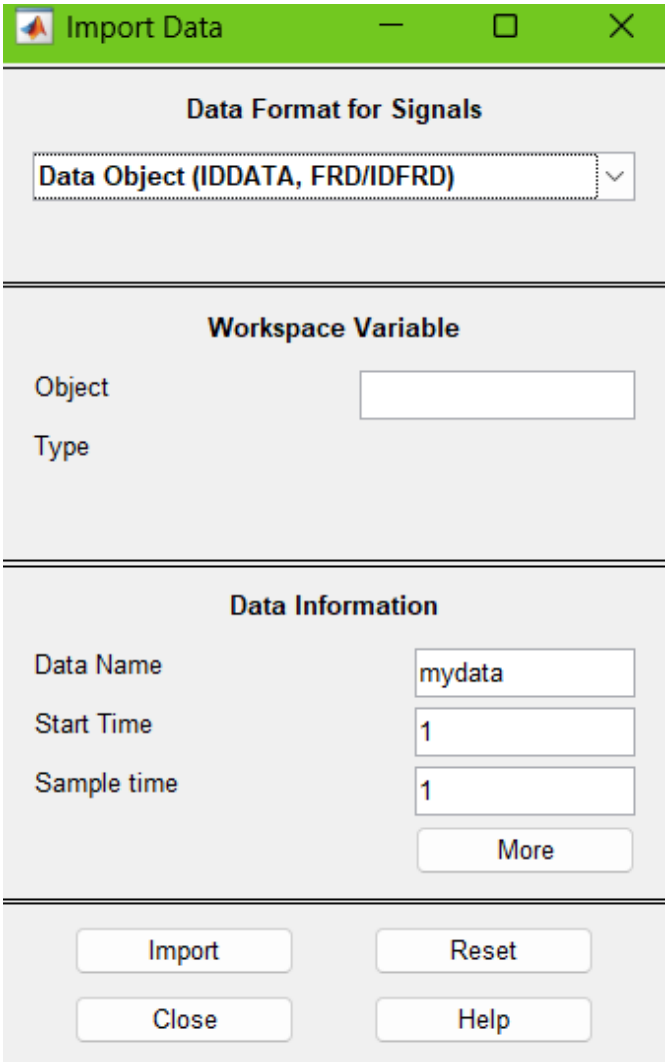


Figura 4: Import data tab

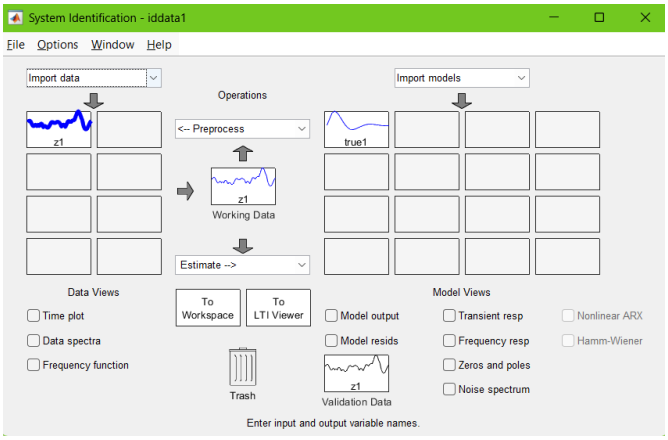


Figura 5: System identification main tab

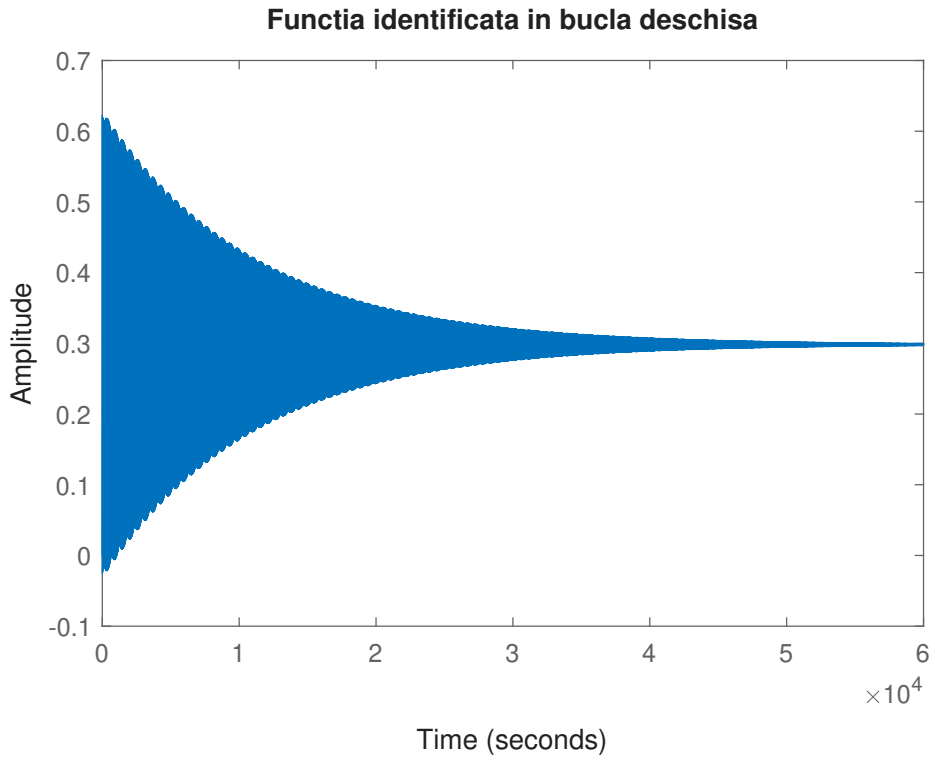


Figura 6: Open-loop of the identified transfer function

After all of this is completed the resulting transfer fucntion may be used to compare with the *iddata* of the validation data (3) and see the fit percentage. For our identification data (2) the transfer function is:

$$H(z) = \frac{0.086497570485291z + 0.110584124642427}{z^2 + 1.788302253458148e - 04z + 0.370708683432494} \tag{1}$$

The resulting graph in open-loop is: We can see that the sterp response is not the same as the one we had in the real world measurements. This is atributted to error of identification, error of measure, over or under sampling etc. The expected transfer fucntion is:

$$H(s) = \frac{K}{s(Ts + 1)} \tag{2}$$

From the identification plot (2) we see almost as an integrator behaviour with saturation at 1.4v and 4.5v. The saturation is expected as there is an end position for both directions of the semi-rotary cylinder. The integrator response is also expected as the valve just opens or closes and if we measure in open-loop, the valve never shuts off, so the rotor just arrives at the end position.

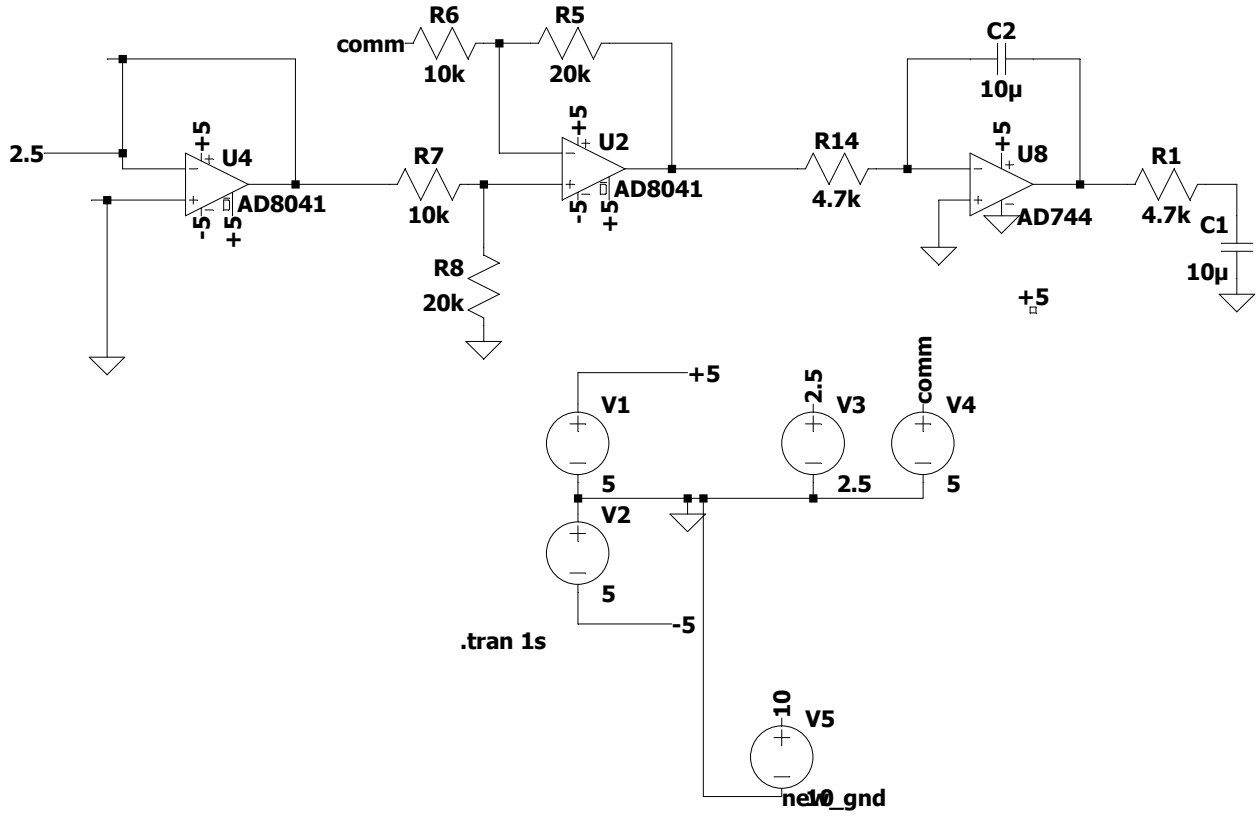
3 Electronic Simulation

After identifying the system in the previous section, we have an understanding of how the simulated system should look like. In this section, we will lay out the steps of trying to emulate the system electronically.

3.1 Design of the cylinder and proportional valve with the help operational amplifiers

From the datasheet of the valve, it was previously established that the valve requires an electrical command of 0-10v to open or close the two way valve. Because there was no need for high voltages to simulate the valve faithfully, the command takes 5v max and determine the direction of the "flow" by

comparing the input with 2.5v. The circuit has a differential opamp circuit, a repeater and a integrator together with a low-pass filter. The command is represented as *comm* and if the command is below the



--- C:\Users\simon\OneDrive\Documents\LTspice\Draft9.asc ---

Figura 7: LtSpice circuit simulation

2.5v mark, the response will be opposite, because of the formulas of the circuits:

$$V_o = V_i \tag{3}$$

$$V_o = \frac{R_2}{R_1}(V_2 - V_1) \tag{4}$$

$$V_o = -\frac{1}{RC}\int V_i dt \tag{5}$$

The order of the system would be the following:

1. get the half-point reference trough a repeater because of the benefits of high impedance (3)
2. get the "direction" of the input by applying the differential formula (4)
3. integrate the direction, to note is that the output is reversed so thats why we get the effect that will be seen later on (5)

3.2 Design of the position measurement

With the help of an arduino ,two **CD405xB** multiplexers and some leds, we hope two display the position of the "rotor". The arduino will take the voltage output of the integrator and tranform the ADC reading into a command for the 3 inputs for each MUX, while also enabling or disabling the MUX that isnt supposed to be turned on.

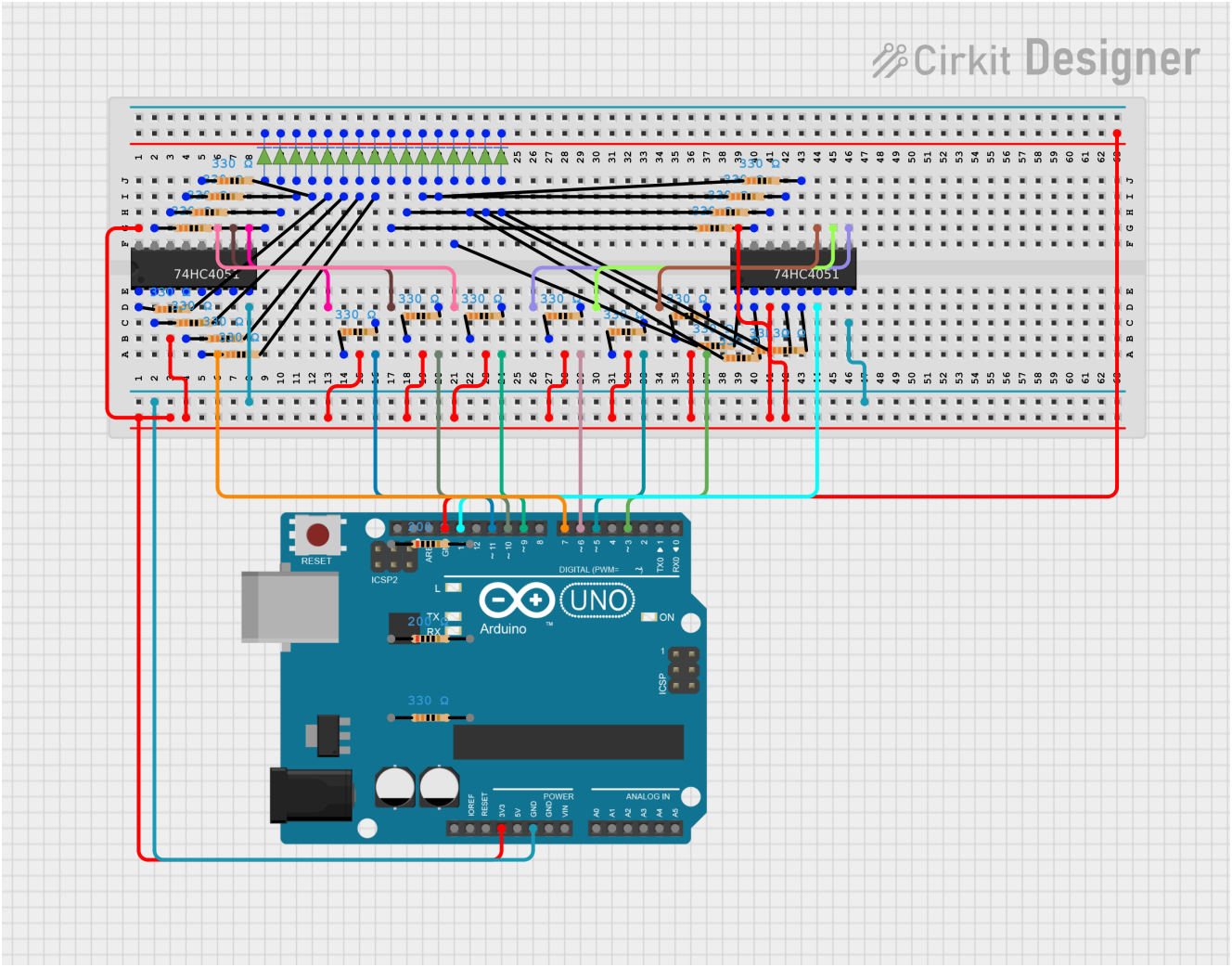


Figura 8: Cicrcuit for position displaying with Arduino UNO

3.3 Control implementation of the electronic simulation

For implementing the control the **AVR32DA32** eval board was used with the following control code:

```
while true do
  if getReadComm() then
    ADC_measurement ← ADC_get_conversion(channel=17)
    writeOneByte(0x15)
    writeOneByte(high byte of ADC_measurement)
    writeOneByte(low byte of ADC_measurement)
  else if getEnableController() then
    if prev = true then
      for i = 2 to 1 do
        c[i] ← 0
        e[i] ← 0
      end for
      prev ← false
    end if
    Toggle PORTF pin 2
    ADC_measurement ← ADC_get_conversion(17)
    realValue ← (ADC_measurement / 1023.0) * 4.6
    e[0] ← getStepVal() − realValue
    writeOneByte(0x0B)
```

```
    aux ← getFirst4digits(e[0])
    control_voltage ← 0
    c[0] ← e[0] · 2.5
    if c[0] > 2 then
      c[0] ← 2
    else if c[0] < −2 then
      c[0] ← −2
    end if
    if c[0] < 0 then
      control_voltage ← 2.1
    else
      control_voltage ← 0
    end if
    control_voltage ← 2.1
    duty ← ((−1 · c[0] + control_voltage) · 0xFFF) /5
    if duty ¿ 0xFFF then
      duty ← 0xFFF
    end if
    PWM_load_duty_cycle(duty)
    if getDataOption() = 0x0 then
      writeOneByte(high byte of duty)
      writeOneByte(low byte of duty)
    else if getDataOption() = 0x1 then
      writeOneByte(high byte of ADC_measurement)
      writeOneByte(low byte of ADC_measurement)
    else if getDataOption() = 0x2 then
      writeOneByte(high byte of ADC_measurement)
      writeOneByte(low byte of ADC_measurement)
    else
      writeOneByte(high byte of duty)
      writeOneByte(low byte of duty)
    end if
    for i = 2 to 1 do
      c[i] ← c[i − 1]
      e[i] ← e[i − 1]
    end for
  else
    prev ← true
  end if
  Delay 2 milliseconds
end while
```

The system itself is a bit complex for the scope of this paper. There are a lot of custum made IRQ functions and hadlers, but the noteworthy thigs foe the code snipet are the control recurrence equations and the snipets where the board sends data back trough UART. The first part is only for the read of the ADC and sending of data on UART, the second if that verifies the getEnableController() function call is where the controller code starts. There are the c[] and e[] vectors that are the command and error vectors, and the $c[0] = e[0] * 2.5$ is the command rule. The next part of the code is the saturation and the command itself that is *duty*. The board has a PWM generator that is connected to a low-pass filter to get a **DAC**, so the board can send a continuous command to the system. The command rule was determined after the system was simulated with the help of *Simulink*, where the transfer functions of all the operational amplifiers were put together. The transfer fucntions used are:

$$H_{integrator} = -\frac{1}{RCs} \tag{6}$$

$$H_{filter} = \frac{1}{RCs + 1} \tag{7}$$

Where:

$$R = 4.7k\Omega$$

$$C = 10\mu F$$

The resulted plot in Matlab is:

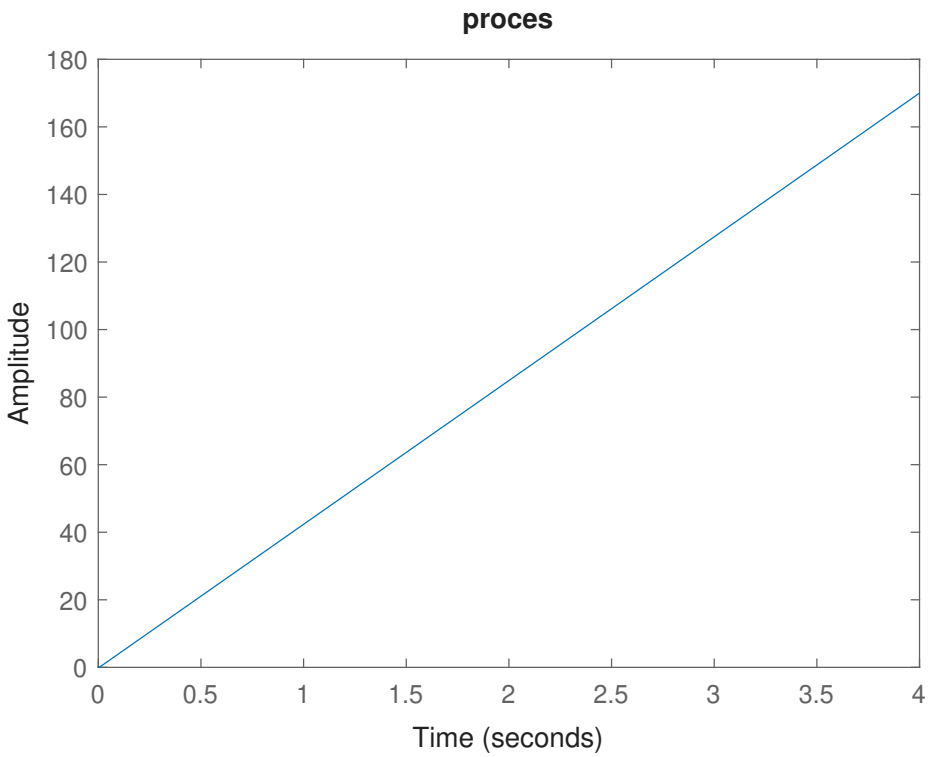


Figura 9: Integrator and filter transfer fucntions in series

And the Simulink model is:

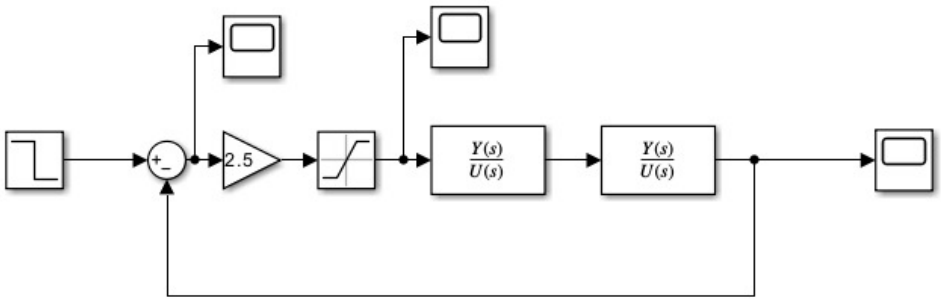


Figura 10: Simulink

In the Simulink model the saturations were also simulated and scopes on command output were added for debugging. The resulted graphs for the command output and proces output are:

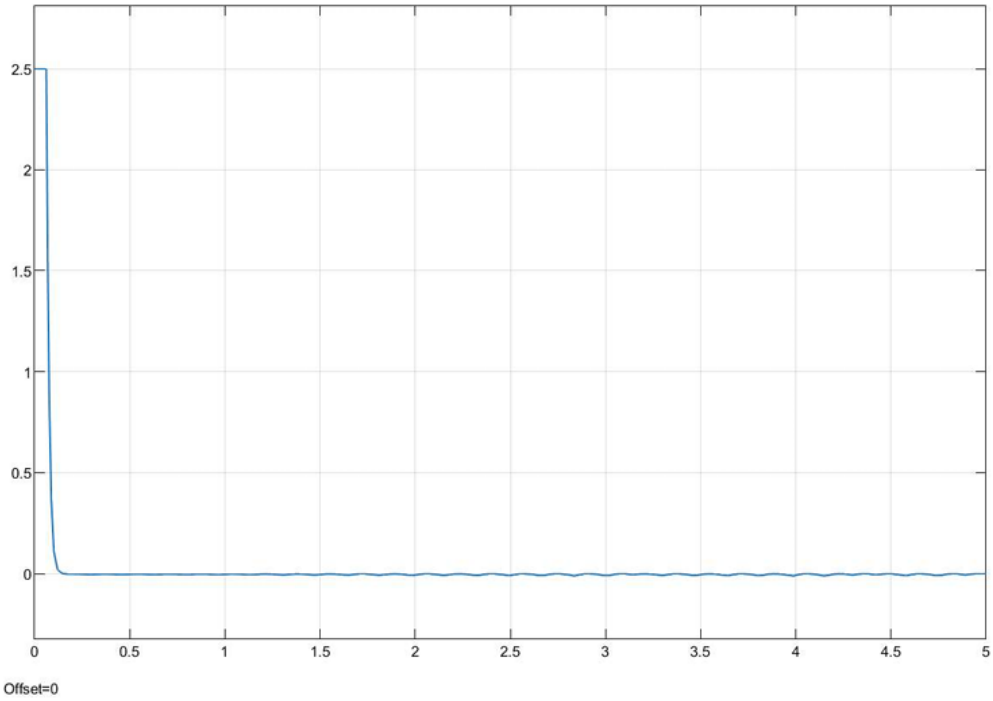


Figura 11: Controller command output scope

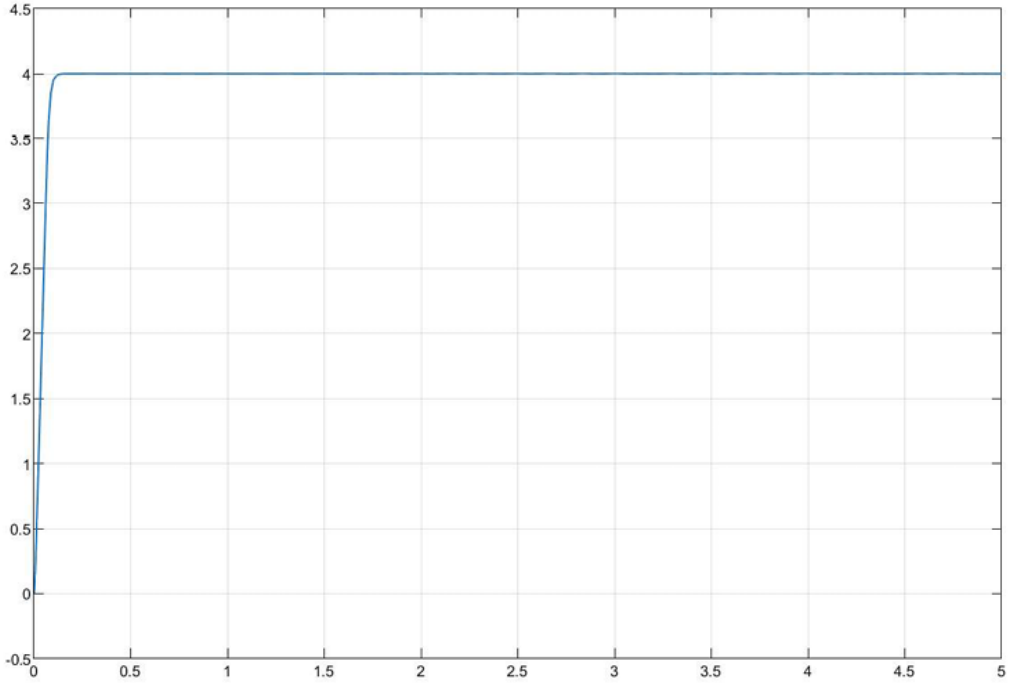


Figura 12: Process output scope

4 Test results

After acquiring all the components and assembling all of them together and running some test for different step values, we get the following results:

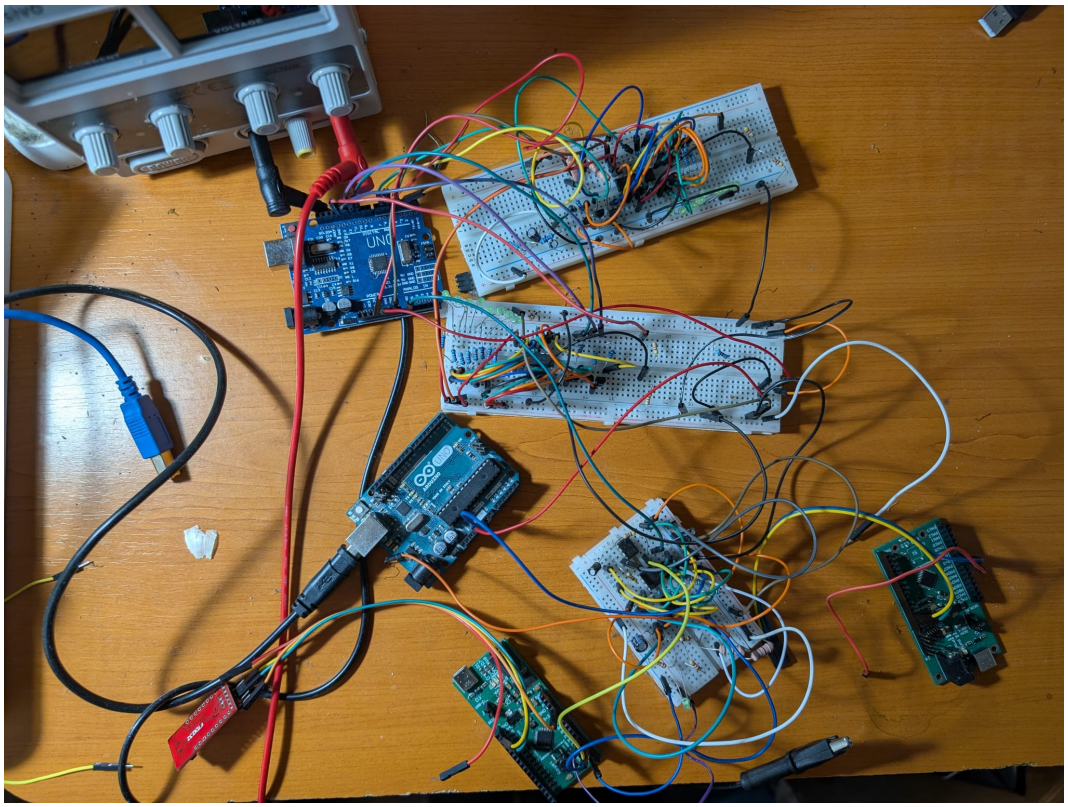


Figura 13: Full circuit

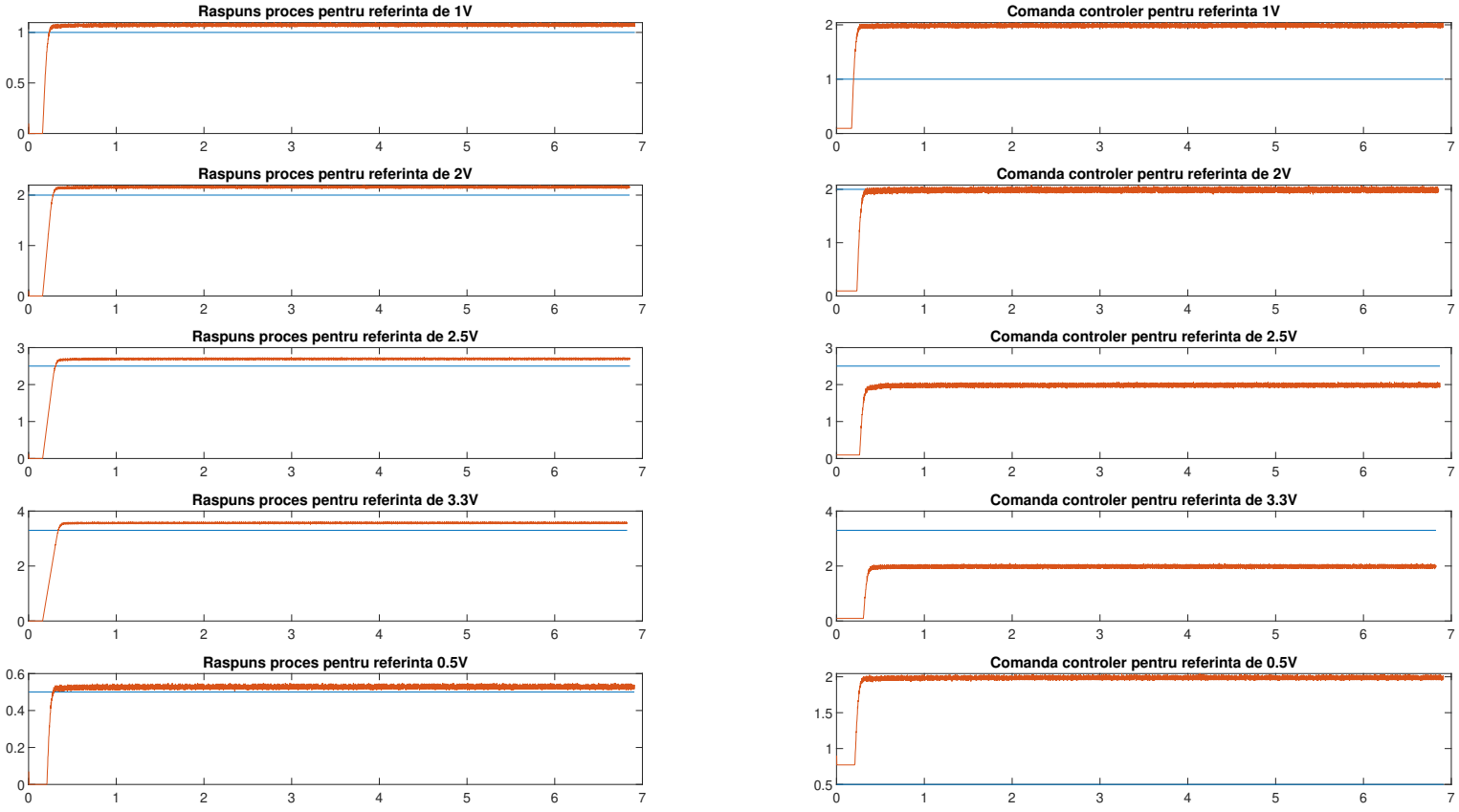


Figura 14: Graph for process output and controll commands for different step values